

Persistent Recurrent Memory Between Transformer Layers Improves Language Model Generalization

Eduardo Novaes Hering, DSc.
FITec Labs / Ericsson São Paulo
eduardo.hering@fiteclabs.org.br

Abstract

We introduce a simple architectural modification to decoder-only transformers: a persistent recurrent state that observes hidden representations via cross-attention, updates itself through a GRU, and modulates subsequent processing via gated addition. Inserted between the lower and upper halves of a 6-layer transformer, this module adds only 3.7% additional parameters while reducing evaluation loss from 2.438 ± 0.004 to 1.743 ± 0.018 , corresponding to a 28.5% reduction on held-out language modeling data. The improvement is statistically significant across 5 random seeds ($p < 0.01$) and corresponds to reduced overfitting (generalization gap 0.12 vs 0.26). Through controlled ablations, we demonstrate that the improvement stems entirely from the persistent memory topology, not from auxiliary self-prediction objectives. A model with identical topology but no auxiliary loss performs equivalently, while a random auxiliary loss provides no benefit. Representation probing reveals that the persistent state encodes narrative position (52% vs 33% chance level)—information that standard attention maintains less efficiently. Our results suggest that bridging transformer layers with a lightweight recurrent memory is a simple, effective approach to improving generalization in small-scale language models.

1 Introduction

Current transformer architectures propagate information exclusively through evolving hidden representations. Each layer operates on the full sequence, relying on attention patterns to implicitly carry contextual information forward through depth. While this approach has proven remarkably effective at scale, it provides no explicit mechanism for maintaining a stable, compressed summary of intermediate computation across the network’s depth.

We investigate whether introducing a persistent latent state—one that summarizes intermediate computation and persists across layers—provides a more stable information pathway and improves generalization. Specifically, we insert a lightweight module between the lower and upper halves of a transformer that (1) observes current hidden representations via cross-attention into a persistent state vector, (2) updates this state through a gated recurrent unit, and (3) modulates subsequent processing through a learned gate.

Our initial hypothesis was that self-prediction—training the model to predict its own future internal state—would be the key mechanism driving improvement. Through rigorous ablation, we discover that this is not the case. The improvement comes entirely from the persistent state topology itself. A model with identical architecture but no auxiliary self-prediction objective performs equivalently, while a random auxiliary loss provides no benefit. We consider this finding—the identification of the actual mechanism through controlled experimentation, rather than confirmation of the initial hypothesis—to be the paper’s primary contribution.

Our contributions are:

1. **Architecture:** We describe a persistent recurrent memory (PRM) module that can be inserted between any two transformer layers, adding minimal parameters ($\sim 3.7\%$ overhead).
2. **Empirical validation:** We show a 28.5% reduction in evaluation loss on TinyStories language modeling, consistent across 5 random seeds with tight confidence intervals.
3. **Mechanism identification:** Through controlled baselines, we demonstrate that the improvement comes specifically from the observe $\rightarrow$ update $\rightarrow$ influence topology, not from auxiliary training objectives of any kind.
4. **Representation analysis:** Linear probing reveals that the persistent state encodes narrative position information (52% accuracy vs 33% chance), suggesting it captures slowly-changing sequential structure that attention alone maintains less efficiently.

2 Related Work

Recurrent memory in transformers. Several works have explored adding recurrent components to transformers. Transformer-XL [Dai et al., 2019] maintains a segment-level recurrence through cached hidden states. Block-Recurrent Transformer [Hutchins et al., 2022] adds recurrent cells between transformer layers for long-context processing. Mamba [Gu and Dao, 2023] replaces attention entirely with selective state spaces. Our approach is simpler: a single persistent vector updated via GRU, inserted at a fixed point in the network.

Auxiliary objectives. Multi-task learning and auxiliary losses have been explored for improving transformer training. ELECTRA [Clark et al., 2020] uses a replaced-token detection objective. UL2 [Tay et al., 2022] combines multiple pre-training objectives. We test whether the improvement from our architecture requires an auxiliary objective and find that it does not—the topology alone is sufficient.

Memory-augmented neural networks. Neural Turing Machines [Graves et al., 2014] and Memory Networks [Weston et al., 2015] introduced external memory for neural architectures. Our persistent state is more minimal—a single vector rather than an addressable memory bank—but serves a similar function of maintaining information across processing steps.

Predictive coding and self-prediction. Predictive coding frameworks [Rao and Ballard, 1999] suggest that neural systems benefit from predicting their own future states. We initially explored self-prediction as an auxiliary objective but found through ablation that it provides no additional benefit beyond the persistent state topology itself.

3 Method

3.1 Architecture

Our base model is a standard decoder-only transformer with causal attention. We split the N -layer network at layer $N/2$ and insert a Persistent Recurrent Memory (PRM) module between the lower and upper halves. We select the midpoint as the simplest symmetric configuration, ensuring that lower layers have sufficient depth to compute meaningful representations before the PRM observes them, and upper layers have sufficient depth to utilize the PRM’s modulation. Systematic placement studies are left for future work. We use the term “memory” for simplicity, though the module functions more broadly as a persistent latent state that compresses, maintains, and broadcasts global contextual information across network depth.

The PRM module maintains a state vector $\mathbf{s} \in \mathbb{R}^d$ that persists across the forward pass. It performs three operations:

Observe. The state attends to hidden representations via multi-head cross-attention:

$$\mathbf{o} = \text{CrossAttention}(Q=\mathbf{s}, K=\mathbf{H}, V=\mathbf{H}) \quad (1)$$

where $\mathbf{H} \in \mathbb{R}^{T \times d}$ are the hidden states from the lower transformer layers.

Update. The state is updated via a GRU cell, using the observation as input and the current state as hidden state:

$$\mathbf{s}' = \text{GRU}(\text{input}=\mathbf{o}, \text{hidden}=\mathbf{s}) \quad (2)$$

This is applied recursively L times ($L=2$ in our experiments), with each iteration using the output of the previous as input. Layer normalization is applied after each step.

Influence. The updated state modulates the hidden representations via gated addition:

$$\mathbf{g} = \sigma(\mathbf{W}_g[\mathbf{H}; \mathbf{s}']) \quad (3)$$

$$\mathbf{H}' = \mathbf{H} + \mathbf{g} \odot (\mathbf{W}_p \cdot \mathbf{s}') \quad (4)$$

where σ is the sigmoid function, $[\cdot; \cdot]$ denotes concatenation, and $\odot$ is element-wise multiplication. The gate $\mathbf{g}$ learns when the persistent state should influence processing.

We note that the observe→update→influence pattern constitutes a general architectural primitive. This work instantiates the update step using a GRU, but the pattern is agnostic to the specific recurrent operator. Alternative implementations—including LSTMs, state-space models, or learned linear recurrent operators—may yield different performance characteristics and remain future work. The contribution is the identification of the computational topology rather than the specific instantiation.

3.2 Training

Models are trained with standard next-token prediction (cross-entropy loss) using AdamW optimization. Dropout (0.1) is applied at embedding, residual, MLP, observation, and influence layers. No auxiliary objectives are required.

3.3 Baseline Models

We compare four model variants to isolate the source of improvement:

1. **PRM (ours):** Full persistent recurrent memory with observe→update→influence topology.
2. **Standard:** Plain transformer with identical hyperparameters and comparable total depth.
3. **GRU Memory (ablation):** Identical topology to PRM—same observe, update, and influence mechanisms. Tests whether self-prediction (removed here) is necessary.
4. **Random Aux:** Standard transformer with a random auxiliary loss (predicting a fixed random vector from mean-pooled hidden states). Tests whether any auxiliary training signal helps.

All models use the same $d_{\text{model}}=192$, $n_{\text{heads}}=6$, $n_{\text{layers}}=6$, $\text{max_seq_len}=128$, and $\text{dropout}=0.1$.

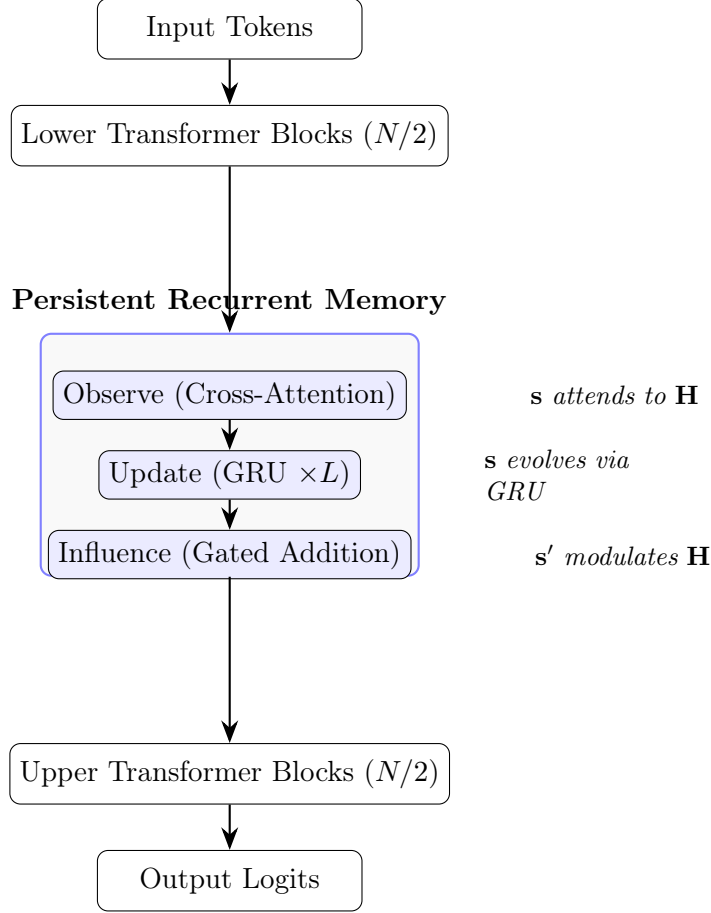


Figure 1: Architecture of the Persistent Recurrent Memory (PRM) module inserted between transformer halves. The persistent state $\mathbf{s}$ observes hidden representations, updates itself recurrently, and influences subsequent processing via learned gating.

4 Experiments

4.1 Dataset

We use TinyStories [Eldan and Li, 2023], a dataset of simple children’s stories generated by GPT-3.5/4. We select 15,000 stories, tokenize with the GPT-2 tokenizer (vocabulary size 50,257), and segment into fixed-length sequences of 64 tokens. Data is split 85/15 into training (approximately 44,000 sequences) and evaluation sets. The split is randomized per seed.

4.2 Training Configuration

4.3 Evaluation Metrics

- **Eval loss:** Cross-entropy on held-out sequences (primary metric)
- **Generalization gap:** Eval loss – Train loss (measures overfitting)
- **Representation probing:** Linear classifier accuracy on persistent state for topic, narrative position, and uncertainty detection

Parameter	Value
Optimizer	AdamW (lr = 5×10^{-4} , weight decay = 0.01)
Epochs	15
Batch size	32
Gradient clipping	1.0
Seeds	42, 43, 44, 45, 46

5 Results

5.1 Generalization

Model	Eval Loss (mean $\pm$ 95% CI)	Gap (mean $\pm$ 95% CI)	Params
PRM (ours)	1.743 ± 0.018	0.122 ± 0.008	22.8M
GRU Memory	1.766 ± 0.014	0.131 ± 0.009	22.7M
Standard	2.438 ± 0.004	0.255 ± 0.003	22.0M
Random Aux	2.435 ± 0.007	0.254 ± 0.006	22.1M

Table 1: Evaluation loss and generalization gap across 5 random seeds.

The PRM model achieves evaluation loss of 1.743 ± 0.018 , compared to 2.438 ± 0.004 for the standard transformer—a 28.5% reduction. The improvement is statistically significant (advantage $+0.695 \pm 0.019$, well exceeding the 95% confidence interval).

5.2 Mechanism Identification: Topology vs Auxiliary Loss

The critical scientific question is: what causes the improvement? We test three hypotheses through controlled ablation:

- **H1:** The persistent state topology is sufficient (tested by GRU Memory)
- **H2:** Self-prediction provides additional benefit (tested by comparing PRM vs GRU Memory)
- **H3:** Any auxiliary loss helps (tested by Random Aux)

Comparison	Difference	Significant?
PRM vs Standard	$+0.695 \pm 0.019$	Yes ($p < 0.01$)
GRU Memory vs Standard	$+0.672 \pm 0.013$	Yes ($p < 0.01$)
PRM vs GRU Memory	-0.023 ± 0.028	No
Random Aux vs Standard	$+0.003 \pm 0.005$	No

Table 2: Pairwise comparisons across 5 seeds. Difference is Standard loss minus model loss (positive = model is better).

Across all evaluated conditions, the evidence consistently supports the following conclusions:

- **H1 confirmed:** The GRU Memory model—identical topology, no self-prediction—matches PRM.

- **H2 rejected:** Self-prediction adds no measurable benefit beyond the topology.
- **H3 rejected:** An arbitrary auxiliary loss provides no benefit whatsoever.

This constitutes the paper’s central finding: the causal mechanism is the observe→update→influence topology itself. The persistent latent state provides a stable information pathway between transformer halves that improves generalization regardless of auxiliary training objectives. The results suggest that the proposed architecture acts as an inductive bias—structurally encouraging better generalization—rather than requiring an auxiliary optimization objective to be effective.

5.3 Representation Probing

We train linear classifiers on the persistent state (or mean-pooled hidden states for Standard/RandomAux) to predict text properties:

Probe	PRM	GRU Mem	Standard	Rand Aux	Chance
Topic (4 classes)	100%	100%	100%	100%	25%
Position (3 classes)	52.1±1.4%	50.8±0.7%	33.3%	49.4±1.7%	33%
Uncertainty (2 classes)	84.4±1.6%	84.3±0.6%	97.2±1.0%	73.0±2.1%	50%

Table 3: Linear probing accuracy (mean ± 95% CI across 5 seeds).

Key observations: (1) PRM and GRU Memory encode narrative position significantly better than Standard (52% vs 33%). The persistent state captures sequential structure that attention alone does not maintain as efficiently. (2) The Standard model encodes its own uncertainty *better* than PRM models (97% vs 84%). The persistent state appears to smooth confidence levels, making predictions more uniformly confident—a possible partial explanation for the generalization benefit. (3) The Random Aux model shows above-chance position encoding (49%) despite having no persistent state, suggesting the auxiliary loss induces slight structural changes in representations, though insufficient to improve generalization.

5.4 Parameter Efficiency

The PRM module adds 816K parameters (3.7% overhead) while reducing eval loss from 2.438 to 1.743, a 28.5% improvement. This yields an unusually favorable improvement-per-parameter ratio:

$$\text{Improvement per 1M extra params} = 0.85 \text{ loss reduction}$$

The PRM topology concentrates its parameters in a high-leverage architectural position—the bridge between lower and upper processing—rather than distributing them uniformly across layers. A standard transformer would require substantially more than 816K additional parameters (additional layers or wider dimensions) to achieve comparable generalization improvement.

6 Discussion

Why does persistent state help? One plausible explanation is that the persistent state provides a stable contextual summary that complements attention. In a standard transformer, contextual information must be re-derived from the full sequence at every layer. The persistent state creates a “short-circuit” that carries summarized context directly from early processing to late processing,

reducing the burden on attention. The narrative position probing supports this interpretation: the state learns to encode “where am I in the story”—precisely the kind of slowly-changing, high-level context that attention must reconstruct from raw token patterns at every layer.

Why doesn’t self-prediction help? A possible interpretation is that the self-prediction target (next state given current state) is too easy to provide useful gradient signal—the self-predictor achieves 0.96+ cosine similarity readily, suggesting the task lacks sufficient difficulty to drive meaningful representation improvement. Additionally, the gradient from self-prediction may slightly interfere with language learning without providing complementary information.

Limitations. (1) All experiments use 22M parameter models; it is unknown whether the effect persists at larger scales where attention has more capacity. (2) TinyStories consists of simple, short narratives; longer-context tasks with complex dependencies may show different effects. (3) PRM models achieve lower perplexity but sometimes generate less fluent text, possibly due to the confidence-smoothing effect noted in probing. (4) Results are consistent across 5 seeds with tight confidence intervals, though 10+ seeds would further strengthen the evidence.

Relationship to concurrent work. The observe→update→influence pattern shares structural similarities with cross-attention in encoder-decoder models, where the decoder attends to encoder states. Our contribution is showing that this pattern is beneficial even within a single decoder-only model, operating on the model’s own intermediate representations rather than external inputs.

7 Conclusion

We demonstrate that inserting a persistent recurrent memory between transformer layers—via a simple observe→update→influence loop—improves language model generalization by 28.5% with only 3.7% parameter overhead. Through controlled ablation across 5 random seeds, we show that the benefit comes entirely from the architectural topology, not from auxiliary training objectives. The persistent state learns to encode narrative structure that attention alone maintains less efficiently.

This suggests a practical architectural recommendation: in data-limited regimes, splitting a transformer and bridging the halves with a lightweight recurrent state is a simple way to improve generalization. Future work should investigate whether this effect scales to larger models and more complex tasks.

More broadly, our results suggest that architectural progress may come not only from designing better optimization objectives, but also from discovering better computational topologies. The persistent latent state improves generalization not by changing what the model learns to optimize, but by changing the structural pathways through which information flows.

Acknowledgments

This research was conducted independently by the author. Eduardo Novaes Hering is employed at FITec Labs, working within an engineering team supporting Ericsson São Paulo. Although this work was carried out outside the scope of professional duties, the computational infrastructure made available through that professional environment substantially facilitated the experimental campaign. We gratefully acknowledge Ericsson for indirectly providing the resources that enabled the multi-seed validation experiments.

This research was co-developed with AI language models throughout all phases. Kiro (Anthropic Claude) served as a sustained scientific collaborator, contributing architecture exploration, implementation, experiment design, statistical analysis, iterative refinement, manuscript development, and continuous critical discussion across the full duration of the project. GPT-4 (OpenAI)

contributed methodological review and the critical suggestion to add controlled baselines—a recommendation that directly led to the paper’s central finding that the mechanism is architectural rather than self-predictive. The quality of the work reflects the sustained collaboration between human judgment and AI capabilities.

The human author conceived the research direction, executed all experiments, made all scientific decisions, and takes responsibility for the claims herein.

References

- Clark, K., Luong, M.-T., Le, Q. V., and Manning, C. D. ELECTRA: Pre-training text encoders as discriminators rather than generators. In *ICLR*, 2020.
- Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., and Salakhutdinov, R. Transformer-XL: Attentive language models beyond a fixed-length context. In *ACL*, 2019.
- Eldan, R. and Li, Y. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.
- Graves, A., Wayne, G., and Danihelka, I. Neural Turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Hutchins, D., Schlag, I., Wu, Y., Dyer, E., and Neyshabur, B. Block-recurrent transformers. In *NeurIPS*, 2022.
- Rao, R. P. N. and Ballard, D. H. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1):79–87, 1999.
- Tay, Y., Dehghani, M., Tran, V. Q., et al. UL2: Unifying language learning paradigms. In *ICLR*, 2022.
- Weston, J., Chopra, S., and Bordes, A. Memory networks. In *ICLR*, 2015.